

MATH3881/5881: Statistical Machine Learning Theory

Week 3: Optimisation in Machine Learning: Foundations

Sarat Moka

UNSW, Sydney

Key Topics

3.1	Foundations of Optimisation	3-2
3.2	Convex Optimisation in Machine Learning	3-6
3.3	Computational Graphs and Automatic Differentiation	3-10
3.4	Backpropagation as Reverse-Mode Autodiff	3-13
3.5	Objective Landscapes in Neural Networks	3-16
3.6	Vanishing and Exploding Gradients	3-18

Books:

(A) *Mathematical Engineering of Deep Learning* book by Liquet, Moka, and Nazarathy (2024).

3.1 Foundations of Optimisation

This week is about how machine learning models are *trained*: how parameters are adjusted to minimise a loss. Before we can prove convergence results for gradient descent, we need a small amount of optimisation vocabulary. This section collects the definitions that will be used throughout Week 3 and Week 4. The treatment closely follows Chapter 4 of *Mathematical Engineering of Deep Learning* (Liquet, Moka, and Nazarathy, 2024).

3.1.1 Formulation

A generic unconstrained optimisation problem has the form

$$\underset{\theta \in \mathbb{R}^d}{\text{minimise}} \quad C(\theta), \quad (3.1)$$

where $C : \mathbb{R}^d \rightarrow \mathbb{R}$ is the **objective function** and $\theta \in \mathbb{R}^d$ is the parameter vector. A point $\theta^* \in \mathbb{R}^d$ is a **(global) minimiser** if

$$C(\theta^*) \leq C(\theta) \quad \text{for all } \theta \in \mathbb{R}^d,$$

and the set of all minimisers is denoted $\arg \min_{\theta} C(\theta)$.

In machine learning, θ collects the model's weights and biases and $C(\theta)$ is the training loss. A maximisation problem is handled by replacing $C(\theta)$ with $-C(\theta)$.

3.1.2 Local Minima, Global Minima, and Saddle Points

A point $\theta \in \mathbb{R}^d$ is a **local minimum** of a continuous function C if there is some $r > 0$ such that

$$C(\theta) \leq C(\phi) \quad \text{for all } \phi \in B(\theta, r), \quad B(\theta, r) := \{\phi \in \mathbb{R}^d : \|\phi - \theta\| < r\}.$$

A **global minimum** satisfies the stronger inequality with ϕ ranging over all of $\mathbb{R}^d$. Every global minimum is a local minimum; the converse fails in general.

If C is differentiable, every local minimum satisfies the **first-order necessary condition**

$$\nabla C(\theta) = 0. \quad (3.2)$$

A point θ at which $\nabla C(\theta) = 0$ is called a **critical point** of C . The condition is necessary but not sufficient: it also holds at local maxima and at saddle points, so a critical point can be any of three things.

To distinguish among them we use second-order information. Recall from Week 0 that the **Hessian** of C at θ is the $d \times d$ symmetric matrix of second partial derivatives,

$$[\nabla^2 C(\theta)]_{i,j} = \frac{\partial^2 C(\theta)}{\partial \theta_i \partial \theta_j}. \quad (3.3)$$

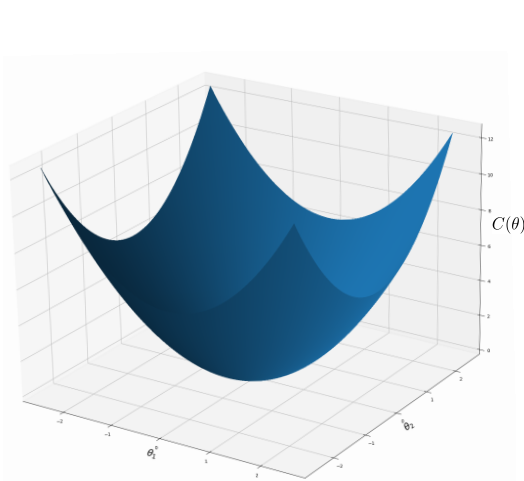
A symmetric matrix $H \in \mathbb{R}^{d \times d}$ is *positive semidefinite* (written $H \succeq 0$) if $\phi^\top H \phi \geq 0$ for all $\phi \in \mathbb{R}^d$, and *positive definite* ($H \succ 0$) if $\phi^\top H \phi > 0$ for all $\phi \neq 0$. Negative semidefinite ($H \preceq 0$) and

negative definite ($H \prec 0$) are defined analogously by reversing the inequality. Equivalently, $H \succeq 0$ iff all eigenvalues of H are ≥ 0 ; $H \succ 0$ iff all eigenvalues are > 0 , and similarly for the negative cases.

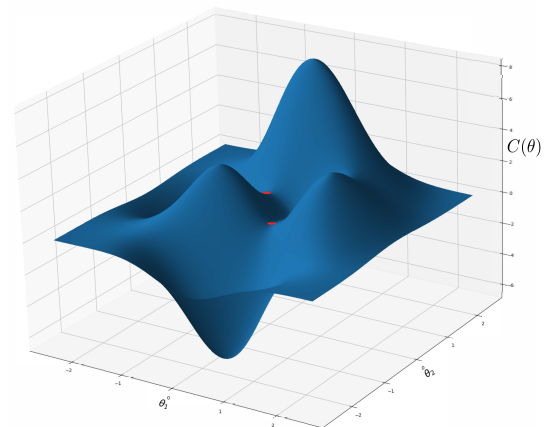
For a twice-differentiable C at a critical point θ (so $\nabla C(\theta) = 0$), the second-order test reads:

- **Local minimum (sufficient condition).** If $\nabla^2 C(\theta) \succ 0$, then θ is a strict local minimum: C increases as we move away from θ in every direction.
- **Local maximum (sufficient condition).** If $\nabla^2 C(\theta) \prec 0$, then θ is a strict local maximum: C decreases as we move away from θ in every direction.
- **Saddle point.** If $\nabla^2 C(\theta)$ has both strictly positive and strictly negative eigenvalues (so $\nabla^2 C(\theta)$ is neither positive semidefinite nor negative semidefinite, also called *indefinite*), then θ is a **saddle point**: C increases along the eigendirection of a positive eigenvalue and decreases along the eigendirection of a negative eigenvalue.

Figure 3.1 illustrates the convex case, a unique global minimum, and the non-convex case, where multiple local minima, local maxima, and saddle points can coexist.



(a) Convex: $C(\theta) = \theta_1^2 + \theta_2^2$ has a unique global minimum at the origin.



(b) Non-convex: the *peaks function* has several local minima, local maxima, and saddle points. Around each strict local minimum the surface is bowl-shaped.

Figure 3.1: Two-dimensional smooth functions illustrating the difference between convex and non-convex objectives.

3.1.3 Convexity

The cleanest setting for optimisation is when C is convex: every local minimum is then a global minimum, and quantitative convergence guarantees become possible. We will rely on three equivalent characterisations of convexity, each suited to a different purpose.

Throughout this section we work in the unconstrained setting $\theta \in \mathbb{R}^d$, consistent with how we use convexity everywhere else in the course.

Zero-order (line-segment) definition. A function $C : \mathbb{R}^d \rightarrow \mathbb{R}$ is **convex** if for all $\theta, \phi \in \mathbb{R}^d$ and $\lambda \in [0, 1]$,

$$C(\lambda\theta + (1 - \lambda)\phi) \leq \lambda C(\theta) + (1 - \lambda)C(\phi). \quad (3.4)$$

Geometrically, the chord joining $(\theta, C(\theta))$ and $(\phi, C(\phi))$ lies on or above the graph of C . C is **strictly convex** if the inequality is strict whenever $\theta \neq \phi$ and $\lambda \in (0, 1)$.

First-order characterisation. If C is differentiable on $\mathbb{R}^d$, then C is convex if and only if

$$C(\phi) \geq C(\theta) + \nabla C(\theta)^\top (\phi - \theta) \quad \text{for all } \theta, \phi \in \mathbb{R}^d. \quad (3.5)$$

That is, the graph of C lies above its linear (first-order) approximation $\phi \mapsto C(\theta) + \nabla C(\theta)^\top (\phi - \theta)$ at every base point θ ; this linear approximation was introduced in Week 0 (Calculus). This is the form used informally in the *Properties of Loss Functions* subsection of the Week 1 notes.

Second-order characterisation. If C is twice differentiable on $\mathbb{R}^d$, then C is convex if and only if

$$\nabla^2 C(\theta) \succeq 0 \quad \text{for all } \theta \in \mathbb{R}^d, \quad (3.6)$$

that is, the Hessian is positive semidefinite everywhere. Strict convexity is equivalent to the Hessian being positive definite ($\nabla^2 C(\theta) \succ 0$) everywhere. This is the characterisation we will use in Section 3.2 to certify the convexity of linear regression as our worked example.

Why convexity matters. If C is convex on $\mathbb{R}^d$, then every local minimum is a global minimum. If C is strictly convex on $\mathbb{R}^d$, the global minimum (when it exists) is *unique*.

For a twice-differentiable C with continuous Hessian, if $\nabla^2 C(\theta^*) \succ 0$ at a critical point θ^* , then $\nabla^2 C(\phi) \succ 0$ on some open ball around θ^* by continuity, and C is convex on that ball. The peaks function in Figure 3.1(b) shows this pattern: around each strict local minimum the surface is bowl-shaped.

3.1.4 Strong Convexity and Smoothness

Two strengthenings of convexity drive the convergence rates we prove in Section 3.2.

Strong convexity. A differentiable function C is μ -**strongly convex** (for $\mu > 0$) if for all θ, ϕ ,

$$C(\phi) \geq C(\theta) + \nabla C(\theta)^\top (\phi - \theta) + \frac{\mu}{2} \|\phi - \theta\|^2. \quad (3.7)$$

Equivalently, for twice-differentiable C , the eigenvalues of $\nabla^2 C(\theta)$ are bounded below by μ . Strong convexity guarantees a unique global minimum.

Smoothness. A differentiable function C is L -**smooth** (for $L > 0$) if its gradient is L -Lipschitz:

$$\|\nabla C(\theta) - \nabla C(\phi)\| \leq L \|\theta - \phi\| \quad \text{for all } \theta, \phi. \quad (3.8)$$

For twice-differentiable *convex* C , this is equivalent to $\nabla^2 C(\theta) \preceq L I$ everywhere. (We only use L -smoothness for convex objectives in this course; for non-convex C the analogous condition is $\|\nabla^2 C(\theta)\|_2 \leq L$, equivalently $-L I \preceq \nabla^2 C(\theta) \preceq L I$.) The key consequence, proved as the Descent Lemma in Section 3.2, is the quadratic upper bound

$$C(\phi) \leq C(\theta) + \nabla C(\theta)^\top (\phi - \theta) + \frac{L}{2} \|\phi - \theta\|^2. \quad (3.9)$$

Condition number. When C is both μ -strongly convex and L -smooth, the ratio

$$\kappa := \frac{L}{\mu} \geq 1$$

is the **condition number** of the problem. We will see in Section 3.2 that κ controls how fast gradient descent converges: small κ means fast convergence, large κ means slow.

3.1.5 Finite-Sum Objectives

Training losses in machine learning almost always take the **finite-sum** form

$$C(\theta) = \frac{1}{n} \sum_{i=1}^n C_i(\theta), \quad (3.10)$$

where n is the size of the training set $\mathcal{D} = \{(x^{(i)}, y^{(i)})\}_{i=1}^n$ and $C_i(\theta)$ is the loss on the i -th sample (squared error, cross-entropy, etc.). When each C_i is twice differentiable, the gradient and Hessian inherit the averaging structure:

$$\nabla C(\theta) = \frac{1}{n} \sum_{i=1}^n \nabla C_i(\theta), \quad \nabla^2 C(\theta) = \frac{1}{n} \sum_{i=1}^n \nabla^2 C_i(\theta). \quad (3.11)$$

Two consequences are used repeatedly in Section 3.2.

- *Convexity is preserved by averaging.* If each C_i is convex, then C is convex (a sum of convex functions is convex).
- *PSD-ness is preserved by averaging.* If $\nabla^2 C_i(\theta) \succeq 0$ for every i , then $\nabla^2 C(\theta) \succeq 0$, because for any $\phi \in \mathbb{R}^d$,

$$\phi^\top \nabla^2 C(\theta) \phi = \frac{1}{n} \sum_{i=1}^n \phi^\top \nabla^2 C_i(\theta) \phi \geq 0.$$

The finite-sum structure also matters *computationally*: when n is large, evaluating $\nabla C(\theta)$ at every iteration is expensive, and we replace the full sum by a single-sample or mini-batch estimate. This is stochastic gradient descent, taken up in Week 4.

3.2 Convex Optimisation in Machine Learning

Equipped with the definitions of Section 3.1, we now study the convex case in detail. This section delivers three things, in order:

- concrete values of L and μ for linear regression as a worked example, certifying that it fits the convex / smooth / strongly convex framework of Section 3.1;
- the *Descent Lemma* (Theorem 3.2.2), the quadratic upper bound which is the workhorse of all L -smooth analysis;
- full proofs of the $O(1/t)$ convergence rate for convex objectives (Theorem 3.2.3) and the linear rate $(1 - \mu/L)^t$ for strongly convex objectives (Theorem 3.2.4), which were stated without proof in the *Convergence rates* remark of the Week 1 notes.

The importance of convexity is twofold: every local minimum is a global minimum, and the structure of the problem allows us to prove *quantitative* convergence guarantees. Understanding this ideal case is essential before studying the non-convex setting of neural networks in Section 3.5.

3.2.1 Convexity of ML Models

We use linear regression as a worked example to verify that the framework of Section 3.1 applies, establishing concrete values of L and μ that will let us read off iteration counts from the convergence theorems proved later in this section.

Linear Regression. With $C(\theta) = \frac{1}{n}\|y - X\theta\|^2$, the Hessian is

$$\nabla^2 C(\theta) = \frac{2}{n} X^\top X.$$

Since $X^\top X$ is positive semidefinite, C is convex by the second-order characterisation (3.6). Writing $\lambda_{\max}$ and $\lambda_{\min}$ for the largest and smallest eigenvalues of $X^\top X$:

- C is L -smooth with $L = 2\lambda_{\max}/n$.
- If X has full column rank, $\lambda_{\min} > 0$ and C is μ -strongly convex with $\mu = 2\lambda_{\min}/n$, giving condition number $\kappa = \lambda_{\max}/\lambda_{\min}$.

Logistic Regression. With augmented inputs $\tilde{x}^{(i)} \in \mathbb{R}^{p+1}$ and labels $y^{(i)} \in \{0, 1\}$, the binary cross-entropy training objective from Week 1 is

$$C(\theta) = -\frac{1}{n} \sum_{i=1}^n \left[y^{(i)} \log \sigma(\theta^\top \tilde{x}^{(i)}) + (1 - y^{(i)}) \log(1 - \sigma(\theta^\top \tilde{x}^{(i)})) \right].$$

This $C(\theta)$ is convex on $\mathbb{R}^{p+1}$ and L -smooth with $L = \lambda_{\max}(X^\top X)/(4n)$, where X is the design matrix from Week 1 (rows $\tilde{x}^{(i)\top}$). It is *not* strongly convex in general, so gradient descent achieves only the $O(1/t)$ rate (Theorem 3.2.3). The Hessian-based proof is in Section 4 of the Week 3 Supplement (optional reading).

Softmax Regression. With K classes, weight matrix $W \in \mathbb{R}^{K \times (p+1)}$, one-hot labels $y^{(i)} \in \{0, 1\}^K$, and softmax outputs $\hat{y}^{(i)} = \text{softmax}(W\tilde{x}^{(i)})$, the categorical cross-entropy training objective from Week 1 is

$$C(W) = -\frac{1}{n} \sum_{i=1}^n \sum_{k=1}^K y_k^{(i)} \log \hat{y}_k^{(i)}.$$

This $C(W)$ is convex in W . As with logistic regression, it is not strongly convex without regularisation, so gradient descent on it achieves only the $O(1/t)$ rate. The proof, via the log-sum-exp function, is in Section 5 of the Week 3 Supplement (optional reading); the same derivations appear in Chapter 4 of *Mathematical Engineering of Deep Learning* (Liquet, Moka, and Nazarathy, 2024).

Remark 3.2.1

Adding L2 regularisation $\frac{\lambda}{2}\|\theta\|^2$ to the squared-error loss above adds λI to the Hessian, making the combined objective λ -strongly convex regardless of the data. The same statement holds for any convex loss with a λ -strongly convex penalty added. This is one reason regularisation improves both optimisation (faster convergence) and generalisation (unique minimiser) simultaneously.

Exercise 3.2.1

Consider ridge regression: $C(\theta) = \frac{1}{n}\|y - X\theta\|^2 + \frac{\lambda}{2}\|\theta\|^2$ with $\lambda > 0$, where $X \in \mathbb{R}^{n \times d}$ has full column rank and $\lambda_{\max}, \lambda_{\min} > 0$ denote the largest and smallest eigenvalues of $X^\top X$.

- (a) Compute the Hessian $\nabla^2 C(\theta)$ and show C is L -smooth and μ -strongly convex with

$$L = \frac{2\lambda_{\max}}{n} + \lambda, \quad \mu = \frac{2\lambda_{\min}}{n} + \lambda.$$

- (b) Show that the condition number satisfies $\kappa(\lambda) = L/\mu \rightarrow \lambda_{\max}/\lambda_{\min}$ as $\lambda \rightarrow 0$ and $\kappa(\lambda) \rightarrow 1$ as $\lambda \rightarrow \infty$. Explain in words why regularisation speeds up gradient descent.
- (c) Let θ^* denote the unique minimiser of C (it exists and is unique because C is μ -strongly convex with $\mu > 0$). Using Theorem 3.2.4, find the number of gradient descent iterations (with $\alpha = 1/L$) needed to guarantee $\|\theta^{(t)} - \theta^*\|^2 \leq \varepsilon \|\theta^{(0)} - \theta^*\|^2$. Express your answer in terms of κ and ε . (*Hint:* use $\log(1 - 1/\kappa) \geq -2/\kappa$ for $\kappa \geq 2$.)
- (d) Suppose $\lambda_{\max}/\lambda_{\min} = 2500$. Without regularisation, estimate the iteration count for $\varepsilon = 10^{-6}$. How does adding $\lambda = 2\lambda_{\min}/n$ change the condition number and iteration count?

Remark 3.2.2 (What about SVMs?)

SVMs are also convex but sit outside this section's framework. The hard-margin form,

$$\min_{w,b} \frac{1}{2} \|w\|^2 \quad \text{subject to} \quad y^{(i)}(w^\top x^{(i)} + b) \geq 1, \quad i = 1, \dots, n,$$

is a *constrained* convex quadratic program (the soft-margin version adds slack variables but keeps the constraints). The unconstrained hinge-loss reformulation in Week 1 is also convex but not L -smooth (the hinge has a kink). Either way, Theorem 3.2.3 and Theorem 3.2.4 below do not apply directly; see the SVM Supplement to Week 1.

3.2.2 The Descent Lemma

The fundamental tool in the analysis of gradient descent for smooth objectives is the following inequality.

Theorem 3.2.1 (Descent Lemma)

Let $C : \mathbb{R}^d \rightarrow \mathbb{R}$ be L -smooth. Then for any $\theta, \phi \in \mathbb{R}^d$,

$$C(\phi) \leq C(\theta) + \nabla C(\theta)^\top (\phi - \theta) + \frac{L}{2} \|\phi - \theta\|^2. \quad (3.12)$$

Proof: See Section 1 of the Week 3 Supplement (optional reading). ■

Applying the descent lemma with one gradient descent step $\phi = \theta - \alpha \nabla C(\theta)$ gives:

$$C(\phi) \leq C(\theta) - \alpha \left(1 - \frac{\alpha L}{2}\right) \|\nabla C(\theta)\|^2. \quad (3.13)$$

For any $\alpha \leq 1/L$, the factor $(1 - \alpha L/2) \geq 1/2 > 0$, guaranteeing a decrease at every step where $\nabla C(\theta) \neq 0$.

3.2.3 Convergence of Gradient Descent: Convex Case**Theorem 3.2.2**

Let C be convex and L -smooth with minimiser θ^* . Gradient descent with step size $\alpha = 1/L$ satisfies

$$C(\theta^{(t)}) - C(\theta^*) \leq \frac{L \|\theta^{(0)} - \theta^*\|^2}{2t}. \quad (3.14)$$

(This is the $O(1/t)$ rate stated, without proof, in the *Convergence rates* remark of the Week 1 notes.)

Proof: See Section 2 of the Week 3 Supplement (optional reading). ■

3.2.4 Convergence of Gradient Descent: Strongly Convex Case

Strong convexity yields a much faster linear convergence rate.

Theorem 3.2.3

Let C be μ -strongly convex and L -smooth with unique minimiser θ^* . Gradient descent with step size $\alpha = 1/L$ satisfies

$$\|\theta^{(t)} - \theta^*\|^2 \leq \left(1 - \frac{\mu}{L}\right)^t \|\theta^{(0)} - \theta^*\|^2. \quad (3.15)$$

(This is the linear rate stated, without proof, in the *Convergence rates* remark of the Week 1 notes.)

Proof: See Section 3 of the Week 3 Supplement (optional reading). ■

Example 3.2.1

Consider the simplest possible smooth strongly convex objective: the one-dimensional quadratic $C(\theta) = \frac{L}{2}\theta^2$ on $\theta \in \mathbb{R}$. The gradient is $\nabla C(\theta) = L\theta$ and the Hessian is L , so C is both L -smooth and L -strongly convex with $\mu = L$. The gradient descent iteration is

$$\theta^{(t+1)} = \theta^{(t)} - \alpha L \theta^{(t)} = (1 - \alpha L) \theta^{(t)},$$

and the behaviour after t steps depends entirely on the contraction factor $|1 - \alpha L|$:

- $\alpha = 1/L$: contraction factor 0, so $\theta^{(1)} = 0$. Gradient descent reaches the minimiser in a single step. This is the *optimal* step size.
- $\alpha = 1/(2L)$: contraction factor $1/2$, so $\theta^{(t)} = (1/2)^t \theta^{(0)}$. Linear convergence at rate $1/2$, much slower than the optimal one-step.
- $\alpha = 2/L$: contraction factor -1 , so $\theta^{(t+1)} = -\theta^{(t)}$ and the iterates oscillate forever between $\pm\theta^{(0)}$ without converging.
- $\alpha > 2/L$: $|1 - \alpha L| > 1$, so the iterates diverge.

The last two bullets show why the range of step sizes that guarantees a decrease, identified analytically in the exercise below, is the open interval $(0, 2/L)$: outside this range the per-step contraction factor is at least 1 in magnitude and gradient descent fails.

Exercise 3.2.2

Recall from the one-step decrease bound (3.13) that a single gradient descent step with learning rate α on an L -smooth objective satisfies

$$C(\phi) \leq C(\theta) - \psi(\alpha) \|\nabla C(\theta)\|^2, \quad \psi(\alpha) := \alpha \left(1 - \frac{\alpha L}{2}\right).$$

The function $\psi(\alpha)$ is the *per-step decrease factor*: a larger value of $\psi(\alpha)$ means a larger guaranteed drop in C at each step.

- Show that $\psi(\alpha)$ is maximised over $\alpha > 0$ at $\alpha^* = 1/L$, giving maximum guaranteed decrease $\frac{1}{2L} \|\nabla C(\theta)\|^2$ per step.
- Show that $\psi(\alpha) > 0$ if and only if $\alpha \in (0, 2/L)$. Hence identify the full range of step sizes for which gradient descent is guaranteed to decrease C at every point where $\nabla C(\theta) \neq 0$.
- For a convex L -smooth function, the convergence bound (3.14) gives $C(\theta^{(t)}) - C(\theta^*) \leq \varepsilon$ after $t \geq L \|\theta^{(0)} - \theta^*\|^2 / (2\varepsilon)$ iterations. How does this iteration count scale if the step size is set to $\alpha = 1/(2L)$ instead of $\alpha = 1/L$? (*Hint*: redo the proof sketch with a general α .)

3.3 Computational Graphs and Automatic Differentiation

The convergence proofs of Section 3.2 assumed we have access to the gradient $\nabla C(\theta)$. In practice, computing this gradient is the central computational task of training any ML model. Three broad approaches exist:

- **Numerical differentiation**: approximate each partial derivative by a finite difference, $\partial C / \partial \theta_i \approx (C(\theta + h e_i) - C(\theta)) / h$ for small h . Simple, but requires one evaluation of C per parameter (so d in total), and is inherently inexact: too large an h gives truncation error, too small an h gives round-off error from floating-point arithmetic.
- **Symbolic differentiation**: derive closed-form algebraic expressions for all partial derivatives, as a computer algebra system (e.g. Mathematica) would. Exact, but the symbolic expression of ∇C can be exponentially larger than the expression for C itself: an effect known as *expression swell*, which makes this approach impractical for deep networks.
- **Automatic differentiation (autodiff)**: apply the chain rule to the *elementary operations* (addition, multiplication, exp, sin, etc.) that make up C , evaluating numerically as we go. Exact up to floating-point round-off, and (for a scalar output) the cost is comparable to one evaluation of C , independent of the number of parameters.

Autodiff is the basis of all modern deep learning frameworks (PyTorch, JAX, TensorFlow). We develop it next.

3.3.1 Computational Graphs

A **computational graph** for a function $C : \mathbb{R}^d \rightarrow \mathbb{R}$ is a graph in which each node represents an intermediate quantity computed from its parents via a single elementary operation (addition, multiplication, exp, sin, and the like). Input nodes are the variables $\theta_1, \dots, \theta_d$ (together with any constants); the output node holds the value $C(\theta)$. The value a_i stored at node i is called the **primal** of that node.

Example 3.3.1

Consider $C(\theta) = \sin(\theta_1\theta_2) + \exp(\theta_2 + \theta_3) - 2\theta_3$ on $\mathbb{R}^3$. We introduce intermediate variables (primals) by reading off the elementary operations one by one:

$$a_1 = \theta_1\theta_2, \quad a_2 = \theta_2 + \theta_3, \quad a_3 = -2\theta_3, \quad a_4 = \sin(a_1), \quad a_5 = \exp(a_2), \quad a_6 = a_3 + a_4 + a_5 = C(\theta).$$

The corresponding computational graph is shown in Figure 3.2.

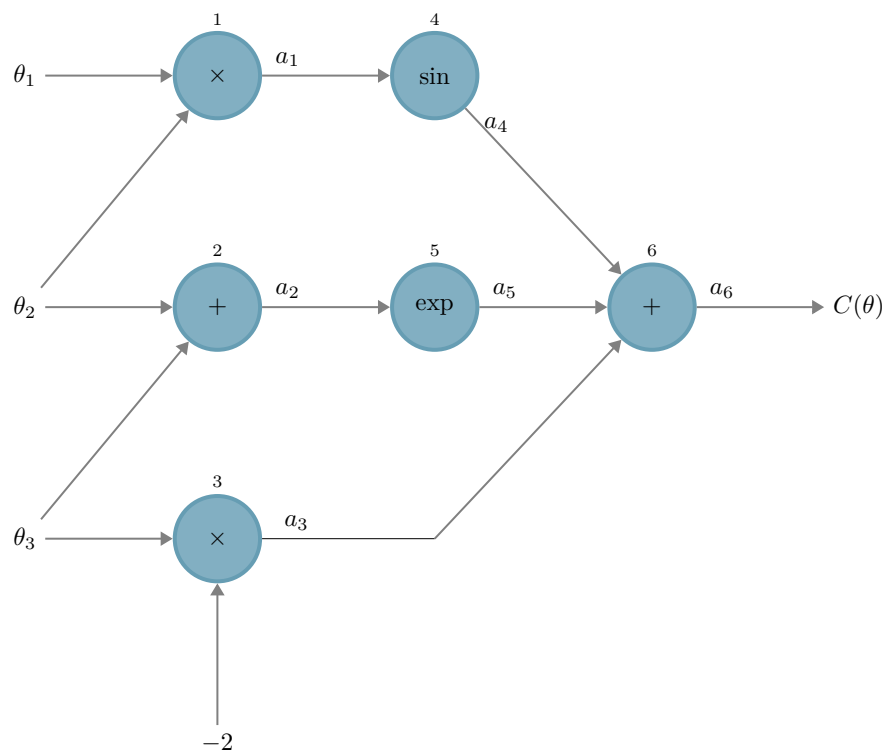


Figure 3.2: Computational graph for $C(\theta) = \sin(\theta_1\theta_2) + \exp(\theta_2 + \theta_3) - 2\theta_3$. Nodes are intermediate computations; edges indicate data flow.

3.3.2 Forward Mode Autodiff

Forward mode computes one partial derivative $\partial C / \partial \theta_j$ in a single forward sweep over the computational graph. Fix one input index j and associate to each node i its **tangent** with respect to θ_j , which is just the partial derivative

$$\dot{a}_i := \frac{\partial a_i}{\partial \theta_j}.$$

The word “tangent” is the standard autodiff term for this partial derivative; it is unrelated to the geometric tangent vectors of Week 0. By the multivariable chain rule applied at node i , if $a_i = f_i(u_1, \dots, u_{\ell_i})$ is computed from its ℓ_i parents $u_1, \dots, u_{\ell_i}$, then

$$\dot{a}_i = \sum_{r=1}^{\ell_i} \frac{\partial f_i}{\partial u_r} \dot{u}_r. \quad (3.16)$$

Initialise with $\dot{\theta}_j = 1$ and $\dot{\theta}_k = 0$ for $k \neq j$, then sweep forward through the graph, evaluating both the primal a_i and the tangent $\dot{a}_i$ at each node. The tangent of the output node is $\partial C / \partial \theta_j$.

Cost. Computing the full gradient $\nabla C(\theta) \in \mathbb{R}^d$ requires d forward sweeps, one per input. This is fine for small d but prohibitive for a neural network with millions of parameters.

3.3.3 Reverse Mode Autodiff

Reverse mode computes the full gradient $\nabla C(\theta) \in \mathbb{R}^d$ in a *single* backward sweep, after one forward sweep has populated the primals. Associate to each node i its **adjoint** (with respect to the output C),

$$\zeta_i := \frac{\partial C}{\partial a_i}. \quad (3.17)$$

The term “adjoint” here is autodiff jargon for this single partial derivative; it is unrelated to the transpose of a matrix. By the multivariable chain rule, if the output of node i is fed into nodes $i_1, \dots, i_{\tilde{\ell}_i}$ (the *children* of node i), then

$$\zeta_i = \sum_{k=1}^{\tilde{\ell}_i} \frac{\partial a_{i_k}}{\partial a_i} \zeta_{i_k}. \quad (3.18)$$

Initialise with $\zeta_{\text{output}} = 1$ at the output node and sweep backwards through the graph; the adjoint of each input variable is the corresponding partial derivative,

$$\frac{\partial C}{\partial \theta_j} = \sum_k \frac{\partial a_{j_k}}{\partial \theta_j} \zeta_{j_k}, \quad (3.19)$$

where the sum is over all nodes j_k that take θ_j as an input.

Example 3.3.2 (Continued)

We evaluate ∇C for $C(\theta) = \sin(\theta_1\theta_2) + \exp(\theta_2 + \theta_3) - 2\theta_3$ at $\theta = (\pi/6, 2, 5)$.

Forward pass (primals):

$$a_1 = \frac{\pi}{3}, \quad a_2 = 7, \quad a_3 = -10, \quad a_4 = \sin\left(\frac{\pi}{3}\right) = \frac{\sqrt{3}}{2}, \quad a_5 = e^7, \quad C = a_6 = \frac{\sqrt{3}}{2} + e^7 - 10.$$

Backward pass (adjoints, initialised with $\zeta_6 = 1$):

$$\begin{aligned} \zeta_3 &= 1, & \zeta_4 &= 1, & \zeta_5 &= 1, \\ \zeta_1 &= \cos(a_1) \zeta_4 = \cos\left(\frac{\pi}{3}\right) = \frac{1}{2}, & \zeta_2 &= e^{a_2} \zeta_5 = e^7. \end{aligned}$$

Gradients (using (3.19)):

$$\frac{\partial C}{\partial \theta_1} = \theta_2 \zeta_1 = 2 \cdot \frac{1}{2} = 1, \quad \frac{\partial C}{\partial \theta_2} = \theta_1 \zeta_1 + \zeta_2 = \frac{\pi}{12} + e^7, \quad \frac{\partial C}{\partial \theta_3} = \zeta_2 - 2 \zeta_3 = e^7 - 2.$$

Remark 3.3.1

Forward vs. reverse mode. For a scalar-valued objective $C : \mathbb{R}^d \rightarrow \mathbb{R}$, forward mode needs d sweeps (one per input) to compute the full gradient ∇C , while reverse mode needs just one forward sweep plus one backward sweep, regardless of d . For neural networks where d is in the millions, reverse mode is overwhelmingly the right choice.

Exercise 3.3.1

Let $C(\theta_1, \theta_2) = \theta_1 \cdot \exp(\theta_1 + \theta_2^2)$. You will compute ∇C at $(\theta_1, \theta_2) = (1, 1)$ using reverse-mode autodiff.

- Introduce intermediate variables $a_1 = \theta_2^2$, $a_2 = \theta_1 + a_1$, $a_3 = \exp(a_2)$, so that $C = \theta_1 \cdot a_3$. Note that θ_1 feeds into *both* a_2 and C directly. Draw the computational graph.
- Perform the forward pass and compute a_1, a_2, a_3, C at $(1, 1)$.
- Initialise $\zeta_C = 1$. Apply the adjoint recursion (3.18) to compute $\zeta_{a_3}, \zeta_{a_2}, \zeta_{a_1}$. Then recover $\partial C / \partial \theta_2$ from ζ_{a_1} (since θ_2 feeds only into a_1). For $\partial C / \partial \theta_1$, sum the contributions of the two places where θ_1 enters: through a_2 (as the additive θ_1 in $a_2 = \theta_1 + a_1$) and directly into C (as the multiplicative θ_1 in $C = \theta_1 \cdot a_3$).
- Verify by differentiating $C(\theta_1, \theta_2) = \theta_1 e^{\theta_1 + \theta_2^2}$ directly and evaluating at $(1, 1)$.

3.4 Backpropagation as Reverse-Mode Autodiff

We now apply the reverse-mode autodiff machinery of Section 3.3 to the computational graph of a feedforward network. The resulting algorithm, which computes the gradient $\nabla_\theta C$ used in the

parameter update $\theta^{(t+1)} = \theta^{(t)} - \alpha \nabla_{\theta} C(\theta^{(t)})$, is known as *backpropagation*.

3.4.1 The FNN as a Computational Graph

The forward pass of an FNN defines the computational graph:

$$x \xrightarrow{\text{Layer 1}} (z^{[1]}, a^{[1]}) \xrightarrow{\text{Layer 2}} (z^{[2]}, a^{[2]}) \rightarrow \dots \rightarrow (z^{[L]}, a^{[L]}) \xrightarrow{\text{Loss}} C.$$

Each layer ℓ performs two steps: the affine map $z^{[\ell]} = W^{[\ell]}a^{[\ell-1]} + b^{[\ell]}$ and the activation $a^{[\ell]} = S^{[\ell]}(z^{[\ell]})$.

The natural per-layer adjoint, following the convention used in *Mathematical Engineering of Deep Learning* (Liquet, Moka, and Nazarathy, 2024), is the gradient of the loss with respect to the pre-activation $z^{[\ell]}$:

$$\delta^{[\ell]} := \frac{\partial C}{\partial z^{[\ell]}} \in \mathbb{R}^{N_{\ell}}, \quad (3.20)$$

called the **delta** (or error signal) for layer ℓ . (Tracking $\delta^{[\ell]}$ rather than the activation-space adjoint $\partial C / \partial a^{[\ell]}$ is the standard simplification because the pre-activation appears in the formulas for the weight and bias gradients below.)

3.4.2 Deriving the Delta Recursion

Output layer ($\ell = L$). The output-layer delta $\delta^{[L]} = \partial C / \partial z^{[L]}$ is computed directly from the loss and the output activation. Two cases arise repeatedly in this course, and both give the clean expression

$$\delta^{[L]} = \hat{y} - y, \quad \text{where } \hat{y} = a^{[L]}. \quad (3.21)$$

(i) *Squared loss with identity output.* If $C = \frac{1}{2} \|a^{[L]} - y\|^2$ and $a^{[L]} = z^{[L]}$, differentiating directly gives $\delta^{[L]} = a^{[L]} - y$.

(ii) *Cross-entropy with softmax output.* If $a^{[L]} = \text{softmax}(z^{[L]})$ with $a_k^{[L]} = e^{z_k^{[L]}} / \sum_j e^{z_j^{[L]}}$, and $C = -\sum_k y_k \log a_k^{[L]}$ with y a one-hot vector (so $\sum_k y_k = 1$), the chain rule gives

$$\delta_k^{[L]} = \frac{\partial C}{\partial z_k^{[L]}} = -\sum_j \frac{y_j}{a_j^{[L]}} \frac{\partial a_j^{[L]}}{\partial z_k^{[L]}}.$$

A direct calculation from the definition of softmax yields the Jacobian entries $\partial a_j^{[L]} / \partial z_k^{[L]} = a_j^{[L]}(\mathbf{1}_{j=k} - a_k^{[L]})$. Substituting and using $\sum_j y_j = 1$,

$$\delta_k^{[L]} = -\sum_j y_j(\mathbf{1}_{j=k} - a_k^{[L]}) = -y_k + a_k^{[L]} \sum_j y_j = a_k^{[L]} - y_k.$$

Hidden layers ($\ell < L$). For $\ell < L$, the node $z^{[\ell]}$ influences C only through $a^{[\ell]}$, which in turn influences C only through $z^{[\ell+1]}$. By the chain rule applied to this two-step path,

$$\delta^{[\ell]} = \frac{\partial C}{\partial z^{[\ell]}} = \left(\frac{\partial a^{[\ell]}}{\partial z^{[\ell]}} \right)^\top \frac{\partial C}{\partial a^{[\ell]}}.$$

The vector activation $a^{[\ell]} = S^{[\ell]}(z^{[\ell]})$ applies a scalar activation $\sigma^{[\ell]}$ elementwise, so its Jacobian is the diagonal matrix $\text{Diag}(\dot{\sigma}^{[\ell]}(z^{[\ell]}))$, where for a vector $v \in \mathbb{R}^n$ we write $\text{Diag}(v)$ for the $n \times n$ matrix whose diagonal is v and whose off-diagonal entries are zero. (Because this matrix is diagonal, it equals its own transpose.) The activation-space gradient is, by the chain rule again, $\partial C / \partial a^{[\ell]} = (W^{[\ell+1]})^\top \delta^{[\ell+1]}$ (since $z^{[\ell+1]} = W^{[\ell+1]}a^{[\ell]} + b^{[\ell+1]}$). Combining,

$$\delta^{[\ell]} = \text{Diag}(\dot{\sigma}^{[\ell]}(z^{[\ell]})) (W^{[\ell+1]})^\top \delta^{[\ell+1]}, \quad \ell = L-1, \dots, 1. \quad (3.22)$$

Gradients with respect to weights and biases. Since $z^{[\ell]} = W^{[\ell]}a^{[\ell-1]} + b^{[\ell]}$, the chain rule one more time gives

$$g_W^{[\ell]} := \frac{\partial C}{\partial W^{[\ell]}} = \delta^{[\ell]} (a^{[\ell-1]})^\top, \quad g_b^{[\ell]} := \frac{\partial C}{\partial b^{[\ell]}} = \delta^{[\ell]}. \quad (3.23)$$

This completes the derivation of backpropagation from first principles. The full procedure is summarised in Algorithm 1.

Algorithm 1: Backpropagation for a fully connected feedforward network

Input: Sample (x, y) , parameters $\theta = \{(W^{[\ell]}, b^{[\ell]})\}_{\ell=1}^L$

Output: Gradients $\{(g_W^{[\ell]}, g_b^{[\ell]})\}_{\ell=1}^L$

- 1 **Forward pass:** set $a^{[0]} = x$; for $\ell = 1, \dots, L$, compute $z^{[\ell]} = W^{[\ell]}a^{[\ell-1]} + b^{[\ell]}$ and $a^{[\ell]} = S^{[\ell]}(z^{[\ell]})$.
 - 2 **Output delta:** $\delta^{[L]} = a^{[L]} - y$ (or as otherwise dictated by the loss).
 - 3 **for** $\ell = L, L-1, \dots, 1$ **do**
 - 4 Set $g_W^{[\ell]} = \delta^{[\ell]} (a^{[\ell-1]})^\top$ and $g_b^{[\ell]} = \delta^{[\ell]}$.
 - 5 **if** $\ell > 1$ **then**
 - 6 Set $\delta^{[\ell-1]} = \text{Diag}(\dot{\sigma}^{[\ell-1]}(z^{[\ell-1]})) (W^{[\ell]})^\top \delta^{[\ell]}$.
-

3.4.3 Computational Cost

The forward pass costs $O(\sum_{\ell} N_{\ell} N_{\ell-1})$ operations, which is the total number of weights in the network; together with the $\sum_{\ell} N_{\ell}$ bias parameters (a lower-order term for typical architectures), this is $O(|\theta|)$, where $|\theta|$ denotes the total parameter count. The backward pass has the same cost order. Hence **one iteration of gradient descent on a single training sample costs** $O(|\theta|)$. For a mini-batch of size n_b , the cost scales to $O(n_b |\theta|)$.

Remark 3.4.1

This is why automatic differentiation transformed deep learning: the cost of computing the *full gradient* is only a small constant multiple of the cost of a *single forward pass*, regardless of the number of parameters. Without reverse-mode autodiff, computing the gradient by finite differences would cost $O(|\theta|^2)$: one forward pass (cost $O(|\theta|)$) per parameter, times $|\theta|$ parameters.

Exercise 3.4.1

Consider a two-layer FNN ($L = 2$) with architecture $N_0 = 2$, $N_1 = 2$, $N_2 = 1$. The hidden layer uses ReLU activation; the output is linear. The loss is MSE: $C = \frac{1}{2}(a^{[2]} - y)^2$ for a scalar target y .

- Write the full forward pass: give explicit equations for $z^{[1]} \in \mathbb{R}^2$, $a^{[1]} \in \mathbb{R}^2$, $z^{[2]} \in \mathbb{R}$, $a^{[2]} \in \mathbb{R}$, and C .
- Show that the output-layer delta is $\delta^{[2]} = a^{[2]} - y \in \mathbb{R}$.
- Apply the delta recursion (3.22) to derive $\delta^{[1]} \in \mathbb{R}^2$ in terms of $W^{[2]}$, $\delta^{[2]}$, and $\dot{\sigma}_{\text{ReLU}}(z^{[1]})$.
- Suppose $z^{[1]} = (-1, 3)^\top$. Write down the specific numerical values of $\dot{\sigma}_{\text{ReLU}}(z_1^{[1]})$ and $\dot{\sigma}_{\text{ReLU}}(z_2^{[1]})$, and simplify $\delta^{[1]}$ accordingly.
- State the dimensions of $g_W^{[2]} = \partial C / \partial W^{[2]}$ and $g_W^{[1]} = \partial C / \partial W^{[1]}$, and write explicit outer-product formulas for each using (3.23).

3.5 Objective Landscapes in Neural Networks

The convergence guarantees of Section 3.2 crucially relied on convexity. For neural networks, the loss landscape is **non-convex**: the composition of non-linear activation functions with matrix products destroys convexity. We now examine the structure of these landscapes.

3.5.1 Non-Convexity of Neural Network Losses

For any neural network with at least one hidden layer and non-linear activations, the loss $C(\theta)$ is non-convex in θ . A simple argument: swapping two neurons in a hidden layer ℓ permutes the corresponding rows of $W^{[\ell]}$ (their input weights) and the corresponding columns of $W^{[\ell+1]}$ (their output weights) but produces exactly the same network output for every input. Thus two distinct parameter vectors realise the same function and therefore have the same loss, but the line segment between them generically passes through higher-loss parameter vectors (where the two neurons have averaged, identical, weights), which violates convexity. The peaks function on the right of Figure 3.1 is a typical low-dimensional caricature of such a landscape: multiple local minima at different heights, saddle points between them, and flat regions where the gradient is small.

3.5.2 Critical Points: Local Minima and Saddle Points

Recall from Section 3.1.2 that a **critical point** is any θ^* with $\nabla C(\theta^*) = 0$. When the Hessian at θ^* is *non-degenerate* (all eigenvalues non-zero), the second-order test classifies it into one of three types: $\nabla^2 C(\theta^*) \succ 0$ (strict local minimum), $\nabla^2 C(\theta^*) \prec 0$ (strict local maximum), or $\nabla^2 C(\theta^*)$ indefinite (saddle point). Degenerate critical points, with one or more zero eigenvalues, are common in deep networks (flat directions, plateaus, ridges) and the second-order test is inconclusive there. For the non-convex landscape of a neural network loss, the practically important observation is:

- In high-dimensional spaces, *saddle points are far more common than local minima*.

Gradient descent is not guaranteed to escape saddle points: a gradient of exactly zero at a saddle will stall the algorithm. In practice, floating-point noise and the gradient variance of stochastic gradient descent (Week 4) act as perturbations that usually allow the algorithm to slip off saddle points after some iterations.

Remark 3.5.1

Benign non-convexity in overparameterised networks. Despite the difficulties above, empirical evidence and a growing body of theory suggest that for sufficiently *overparameterised* networks (many more parameters than training samples), the loss landscape is unexpectedly friendly: most local minima have loss values close to the global minimum, and, in the wide-network regime where weights stay close to their initial values, gradient descent provably converges to a global minimum. The full picture is still an active research area, but in practice, large networks trained with good initialisation, appropriate learning rates, and regularisation reliably converge to solutions that generalise well.

Exercise 3.5.1

This exercise formalises the permutation-symmetry argument for non-convexity.

- Let $P \in \mathbb{R}^{N_1 \times N_1}$ be a permutation matrix ($P^\top P = I$, entries in $\{0, 1\}$). Consider a two-layer FNN with parameters $(W^{[1]}, b^{[1]}, W^{[2]}, b^{[2]})$. Show that replacing $(W^{[1]}, b^{[1]}, W^{[2]})$ by $(PW^{[1]}, Pb^{[1]}, W^{[2]}P^\top)$ produces exactly the same output for every input x .
- Suppose θ^* is a global minimiser of the loss $C(\theta)$. Explain why the $N_1!$ parameter vectors obtained by all permutations of the hidden neurons are also global minimisers with the same loss value.
- Let $N_1 = 2$ and suppose two hidden neurons are genuinely distinct at θ^* (i.e. swapping them gives $\tilde{\theta} \neq \theta^*$). Let $\theta_{\text{mid}} = \frac{1}{2}(\theta^* + \tilde{\theta})$ be the midpoint. Explain why θ_{mid} corresponds to a network in which both neurons have identical (averaged) weights. Using symmetry, argue that such a network is typically *not* a global minimiser, and hence $C(\theta_{\text{mid}}) > C(\theta^*) = \frac{1}{2}(C(\theta^*) + C(\tilde{\theta}))$. Why does this violate convexity?

3.6 Vanishing and Exploding Gradients

Even when gradient descent is running correctly, deep networks suffer from a fundamental numerical problem: gradients can shrink or grow exponentially as they are propagated backward through many layers.

3.6.1 The Jacobian Product Perspective

From the delta recursion (3.22), the delta at layer ℓ is

$$\delta^{[\ell]} = \left(\prod_{k=\ell}^{L-1} \text{Diag}(\dot{\sigma}^{[k]}(z^{[k]})) W^{[k+1]\top} \right) \delta^{[L]}. \quad (3.24)$$

This is a product of $(L - \ell)$ matrices. Define $J^{[k]} = \text{Diag}(\dot{\sigma}^{[k]}(z^{[k]})) W^{[k+1]\top}$; then

$$\delta^{[\ell]} = J^{[\ell]} J^{[\ell+1]} \dots J^{[L-1]} \delta^{[L]}. \quad (3.25)$$

3.6.2 Spectral Norm Analysis

Consider for the analysis a uniform L -layer network: all layers have the same width, the same activation σ , and weight matrices with the same spectral norm $\|W^{[k]}\|_2 = c$ for every k , where $\|\cdot\|_2$ denotes the *spectral norm* of a matrix (its largest singular value, equivalently $\max_{\|v\|=1} \|Mv\|$). Writing $\|\dot{\sigma}\|_\infty := \sup_u |\dot{\sigma}(u)|$ for the maximum absolute value of the activation derivative, submultiplicativity of the spectral norm gives

$$\|J^{[k]}\|_2 \leq \|\dot{\sigma}\|_\infty \cdot c =: \rho,$$

a per-layer expansion bound: applying $J^{[k]}$ to a vector multiplies its norm by at most ρ . Iterating (3.25),

$$\|\delta^{[\ell]}\| \leq \rho^{L-\ell} \|\delta^{[L]}\|. \quad (3.26)$$

- If $\rho < 1$: $\|\delta^{[\ell]}\| \rightarrow 0$ exponentially as ℓ decreases, that is, **vanishing gradients**. Early layers receive negligible gradient signal and effectively stop learning.
- If $\rho > 1$: $\|\delta^{[\ell]}\|$ grows exponentially, that is, **exploding gradients**. Parameter updates become arbitrarily large, destabilising training.

The activation derivative $\dot{\sigma}$ enters ρ through its supremum $\|\dot{\sigma}\|_\infty$:

- **Sigmoid / Tanh**: $|\dot{\sigma}_{\text{Sig}}(u)| \leq 1/4$ and $|\dot{\sigma}_{\text{Tanh}}(u)| \leq 1$, both saturating to zero for large $|u|$. Combined with weight matrices of any reasonable norm, this strongly favours vanishing gradients.
- **ReLU**: $|\dot{\sigma}_{\text{ReLU}}(u)| \in \{0, 1\}$, so the activation derivative does not amplify gradients, and avoids the saturation-driven vanishing issue. The remaining failure mode is the *dead-neuron* effect (see Exercise 3.6.3(d)).

3.6.3 Mitigation Strategies

Three complementary strategies address the vanishing/exploding gradient problem, all introduced in Week 2:

- (i) **Activation function choice:** ReLU avoids saturation-driven vanishing by keeping $\dot{\sigma} = 1$ for all positive inputs.
- (ii) **Weight initialisation:** Xavier and He initialisation (Week 2, Section 5.3) calibrate $\|W^{[\ell]}\|_2$ so that $\rho \approx 1$ at initialisation, preventing exponential growth or decay.
- (iii) **Batch normalisation:** By normalising pre-activations within each mini-batch, batch norm controls the scale of activations dynamically during training, keeping ρ near 1 throughout.

For exploding gradients, **gradient clipping** is an additional fix: rescale the gradient whenever $\|\nabla C(\theta)\|$ exceeds a threshold τ ,

$$\nabla C(\theta) \leftarrow \frac{\tau}{\|\nabla C(\theta)\|} \nabla C(\theta) \quad \text{if } \|\nabla C(\theta)\| > \tau. \quad (3.27)$$

Gradient clipping is widely used in training recurrent networks (Week 7) where long sequences make exploding gradients particularly severe.

Exercise 3.6.1

Consider a uniform L -layer network with the same activation σ and weight matrices $W^{[\ell]}$ all having spectral norm $\|W^{[\ell]}\|_2 = c$.

- (a) Recall from Week 2 that $\sigma'_{\text{Sig}}(u) \leq 1/4$ for all $u \in \mathbb{R}$, with equality at $u = 0$ (an exercise in the Week 2 notes). State this bound; no re-derivation needed.
- (b) Using $\|J^{[k]}\|_2 \leq \|\dot{\sigma}\|_{\infty} \cdot c$ and the Jacobian product formula (3.25), derive the bound

$$\|\delta^{[1]}\|_2 \leq \left(\frac{c}{4}\right)^{L-1} \|\delta^{[L]}\|_2$$

for a network with sigmoid activations.

- (c) For a 20-layer sigmoid network with $c = 1$, compute the numerical value of $(1/4)^{19}$. Is this gradient signal usable in practice? What value of c would be needed to prevent vanishing (keeping the bound ≥ 1)?
- (d) Now suppose ReLU activations are used with the same $c = 1$. What does the Jacobian bound give for $\|\delta^{[1]}\|_2$? Explain why this is an improvement, and identify the remaining failure mode. (*Hint:* consider what happens to a neuron whose pre-activation $z_j^{[\ell]}$ is negative for every training sample. Such a neuron is sometimes called *dead*, because both its output and the gradient flowing into it are exactly zero, so it never wakes up under gradient descent.)

Remark 3.6.1

The vanishing gradient problem was a major obstacle to training deep networks in the 1990s and early 2000s. Its resolution, through ReLU activations (Nair & Hinton, 2010^a), careful initialisation (Glorot & Bengio, 2010^b), and batch normalisation (Ioffe & Szegedy, 2015^c), was a key enabler of the modern deep learning era.

^aV. Nair and G. E. Hinton. *Rectified linear units improve restricted Boltzmann machines*. In International Conference on Machine Learning (ICML), pp. 807–814, 2010.

^bX. Glorot and Y. Bengio. *Understanding the difficulty of training deep feedforward neural networks*. In International Conference on Artificial Intelligence and Statistics (AISTATS), 9:249–256, 2010.

^cS. Ioffe and C. Szegedy. *Batch normalization: accelerating deep network training by reducing internal covariate shift*. In International Conference on Machine Learning (ICML), pp. 448–456, 2015.